



МИНОБРНАУКИ РОССИИ
федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технологический университет «СТАНКИН»
(ФГАОУ ВО «МГТУ «СТАНКИН»)

Институт
Информационных
технологий

Кафедра
Информационных
технологий
и вычислительных систем

ОТЧЕТ О ВЫПОЛНЕНИИ
ЛАБОРАТОРНОЙ РАБОТЫ №1 ПО ДИСЦИПЛИНЕ
«Математическое и компьютерное моделирование»

СТУДЕНТА 1 КУРСА магистратуры ГРУППЫ ИДМ-26-08
(уровень профессионального образования)

ЗАМОСТЬЯНОВА НИКОЛАЯ ВЛАДИСЛАВОВИЧА

НА ТЕМУ
Математическое и компьютерное моделирование на примере модели
"Хищник-Жертва" (Лотки-Вольтерры)

Направление: 09.04.04 Программная инженерия
Профиль Технологии разработки интеллектуальных систем и
подготовки: программных комплексов

Отчет сдан «__» _____ 2026г.

Оценка _____

Преподаватель Палванов Мурод Равшанович, ст. преподаватель
(Ф.И.О., должность, степень, звание.) _____
(подпись)

МОСКВА 2026

ВВЕДЕНИЕ

На замкнутой территории живут два вида: жертвы VVV и хищники PPP. Система изолирована. Хищники питаются жертвами. Численности меняются со временем и зависят от коэффициентов рождаемости, смертности и успешности охоты.

Уравнения:

$$\frac{dV}{dt} = (a - bP)V$$

$$\frac{dP}{dt} = (-c + dV)P$$

Смысл величин:

V — численность жертв;

P — численность хищников;

t — время;

a — рождаемость жертв;

b — смертность жертв при встрече с хищником;

c — убыль хищников при нехватке еды;

d — достаточность еды для размножения хищников.

Если жертв нет ($V=0$), хищники вымирают с скоростью ccc. Если хищников нет ($P=0$), жертвы растут с скоростью a. Встречи учтены произведениями VP.

Цель работы. Построить компьютерную модель, решить систему методом Рунге–Кутты 4 порядка, построить графики, найти равновесие и выяснить влияние a,b,c,d.

Параметры расчёта:

Параметр	Значение
a	0,8
b	0,02
c	0,4
d	0,01

$V(0)$	50
$P(0)$	20
t	от 0 до 80
шаг h	0,05

МЕТОД И БЛОК-СХЕМА

Метод Рунге–Кутты 4 порядка — явный одношаговый метод решения ОДУ.

Для системы $y' = f(y)$, где $y = (V, P)$:

$$k_1 = f(y)$$

$$k_2 = f(y + (h/2)*k_1)$$

$$k_3 = f(y + (h/2)*k_2)$$

$$k_4 = f(y + h*k_3)$$

$$y_{\{n+1\}} = y_n + (h/6)*(k_1 + 2*k_2 + 2*k_3 + k_4)$$

Правая часть от времени явно не зависит.

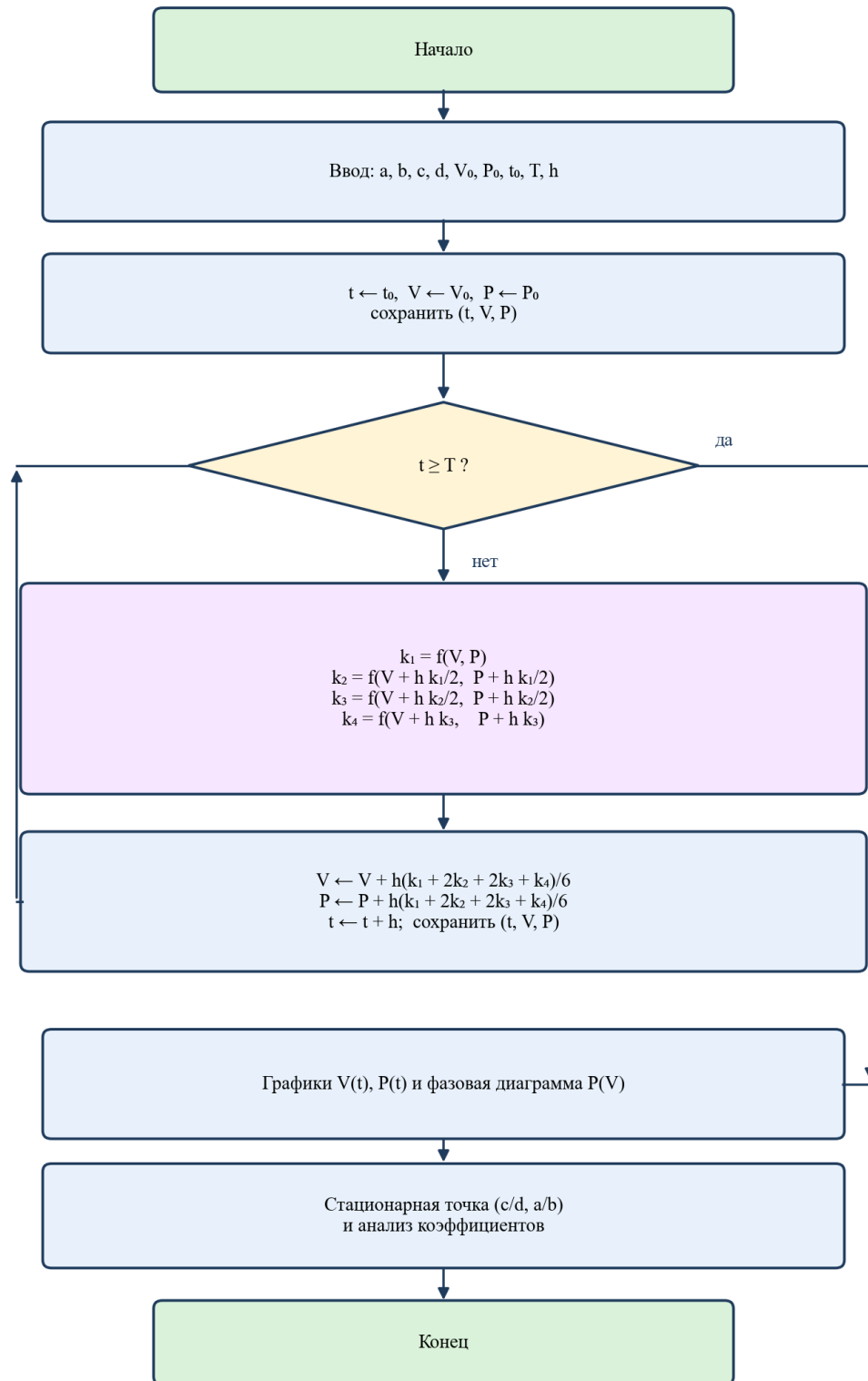


Рис. 1. Блок-схема численного решения системы методом Рунге–Кутты 4-го порядка

ГРАФИКИ И РЕЗУЛЬТАТЫ

Система решена RK4, 1600 шагов.

t	V_prey	P_predator
0.000000	50.00000000	20.00000000
0.050000	51.00742665	20.10530106
0.100000	52.02939252	20.22141364
0.150000	53.06537814	20.34866371
0.200000	54.11479621	20.48739342

Сравнение шагов h и $h/$ даёт погрешность порядка $5.9 \cdot 10^{-6}$ по V и $4.9 \cdot 10^{-6}$ по P .

ДИНАМИКА ПОПУЛЯЦИЙ

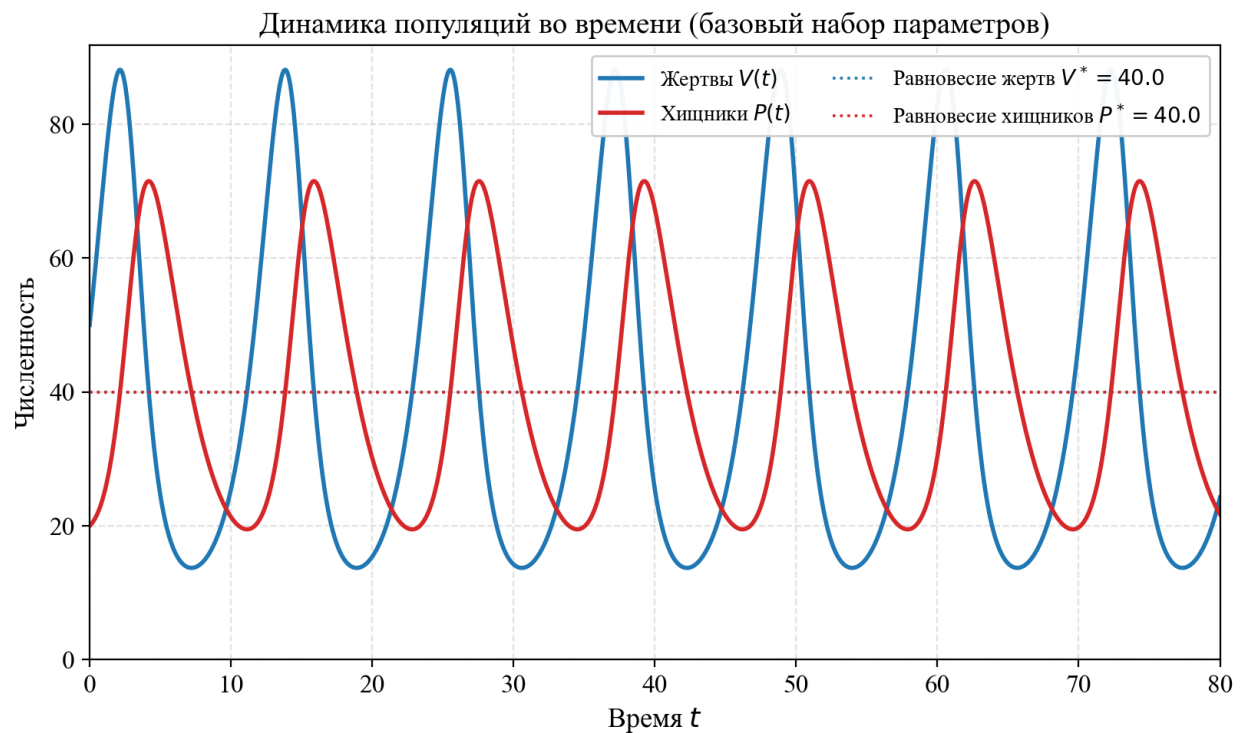


Рис. 2. Изменение численности жертв $V(t)$ и хищников $P(t)$ во времени

Фазовая диаграмма

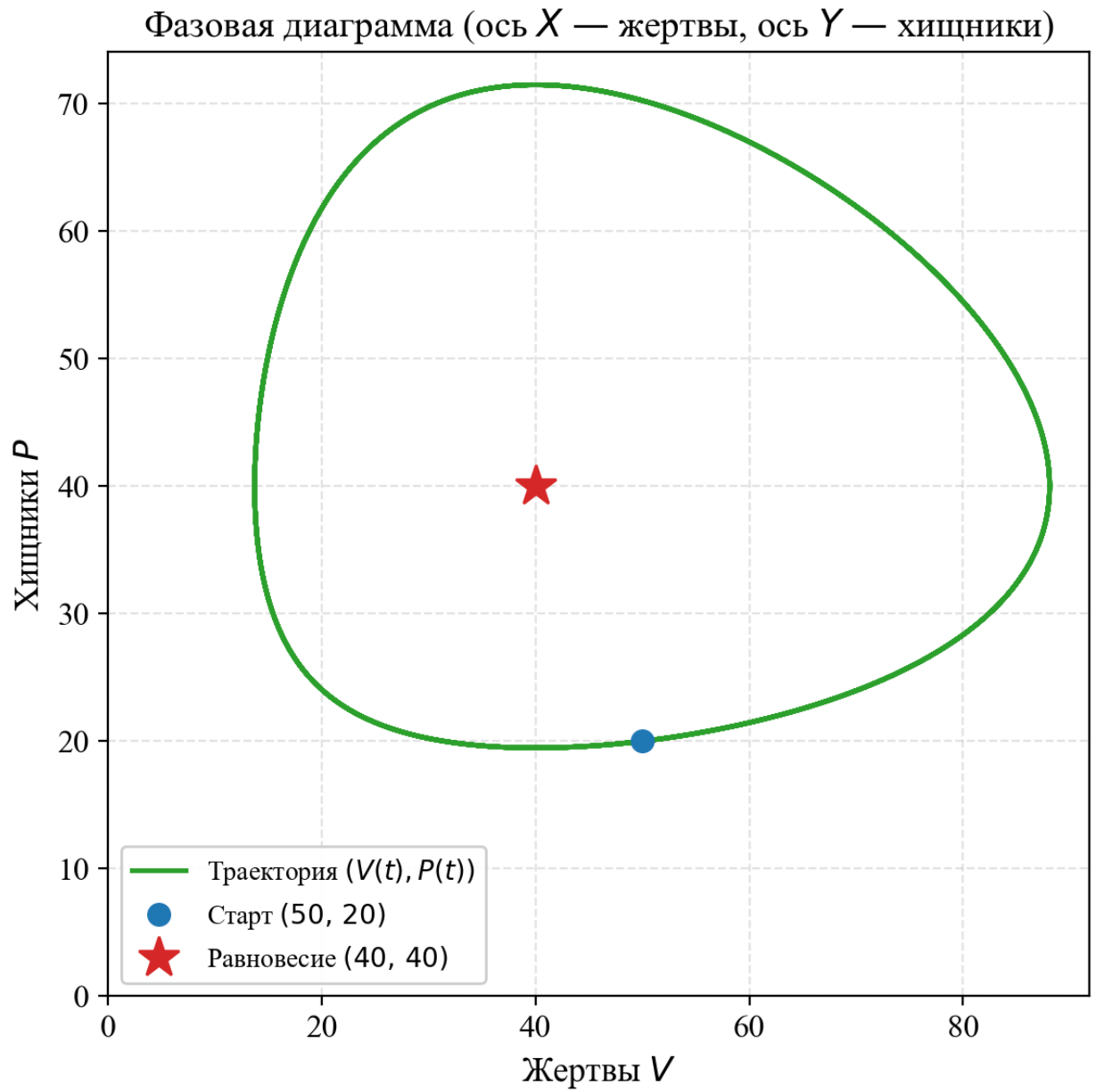


Рис. 3. Фазовая диаграмма системы: ось X — жертвы V , ось Y — хищники P

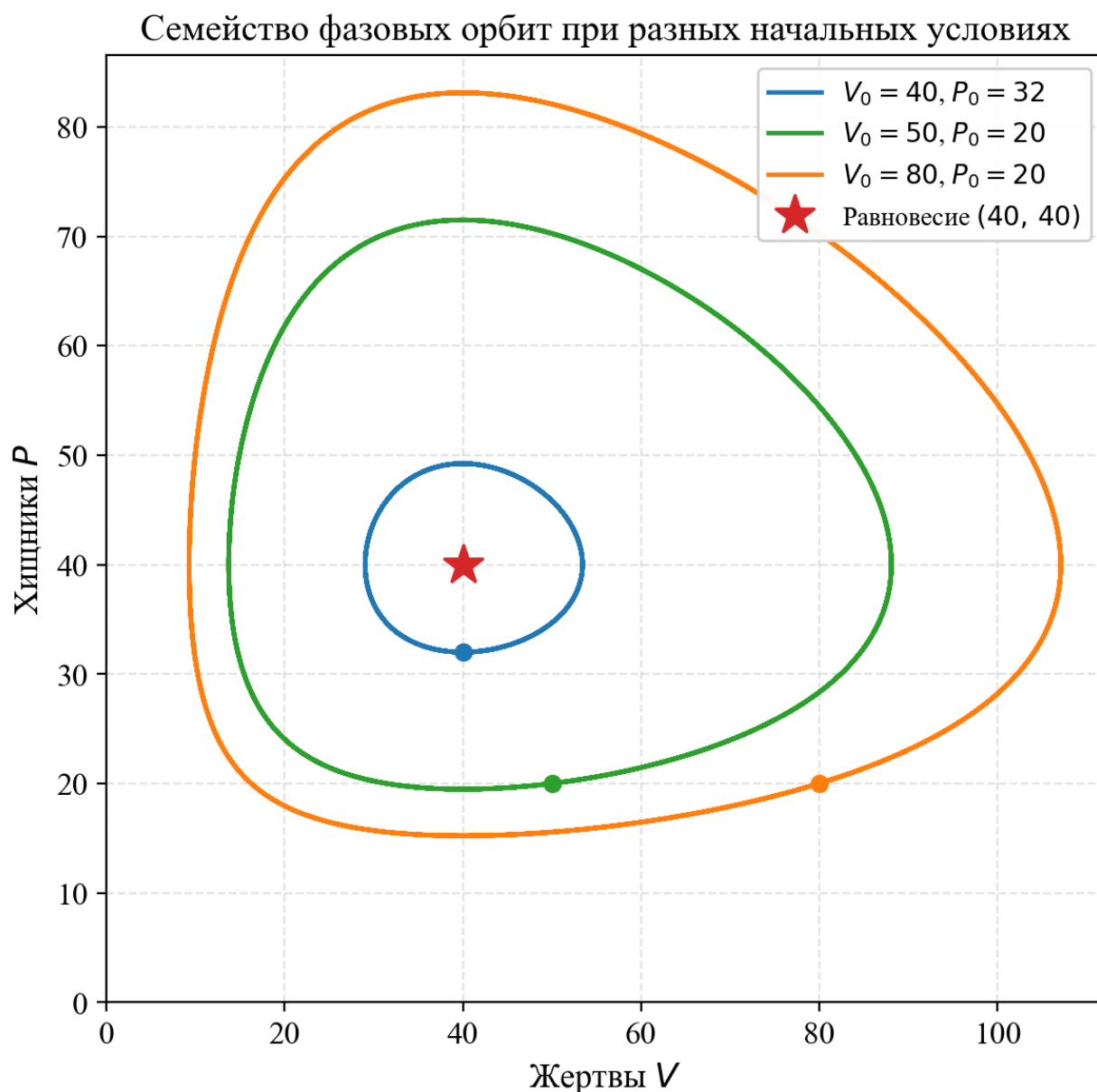


Рис. 4. Семейство фазовых траекторий при различных начальных условиях

На рис. 3 — замкнутая орбита вокруг точки (40; 40), старт (50; 20).

На рис. 4 — вложенные циклы с одним центром.

СТАЦИОНАРНАЯ ТОЧКА

Из условий $dV/dt = 0$ и $dP/dt = 0$:

$$V^*(a - b^*P) = 0 \Rightarrow V=0 \text{ или } P = a/b$$

$$P^*(-c + d^*V) = 0 \Rightarrow P=0 \text{ или } V = c/d$$

Две точки равновесия:

- 1) $(V, P) = (0, 0)$ — вырождение обеих популяций;
- 2) $(V^*, P^*) = (c/d, a/b)$ — сосуществование.

Для базовых коэффициентов: $(V^*, P^*) = (40, 40)$.

Отдельная картинка не нужна: точка уже отмечена звездой на рис. 3 и 4.

Влияние коэффициентов

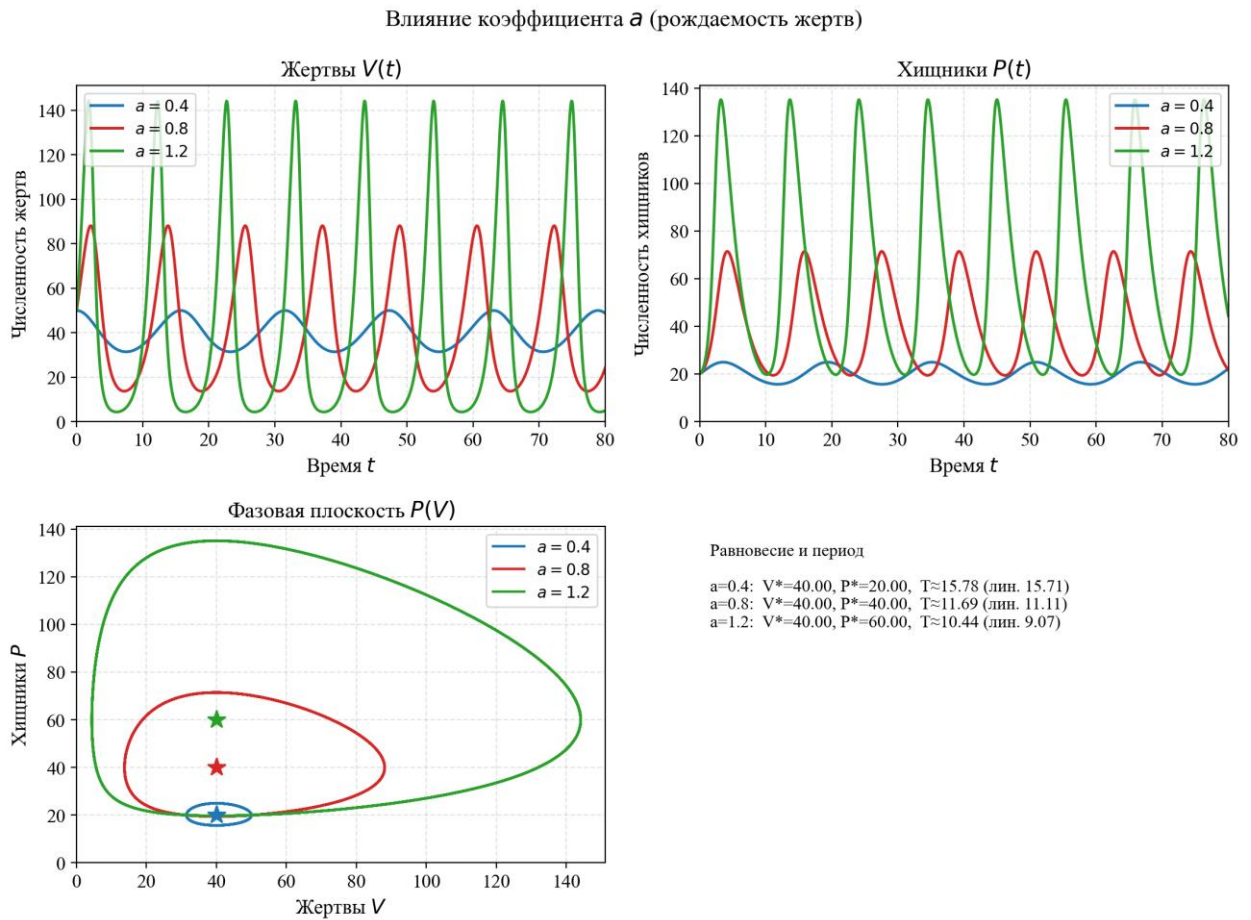


Рис. 5. Влияние коэффициента a (рождаемость жертв) на динамику и фазовый портрет

a	V^*	P^*	T	$V_{\max}$	$T_{\max}$
0,4	40	20	15,78	50	25
0,8	40	40	11,69	88,12	71,49
1,2	40	60	10,44	144,2	135,2

Влияние коэффициента b (смертность жертв при встрече с хищником)

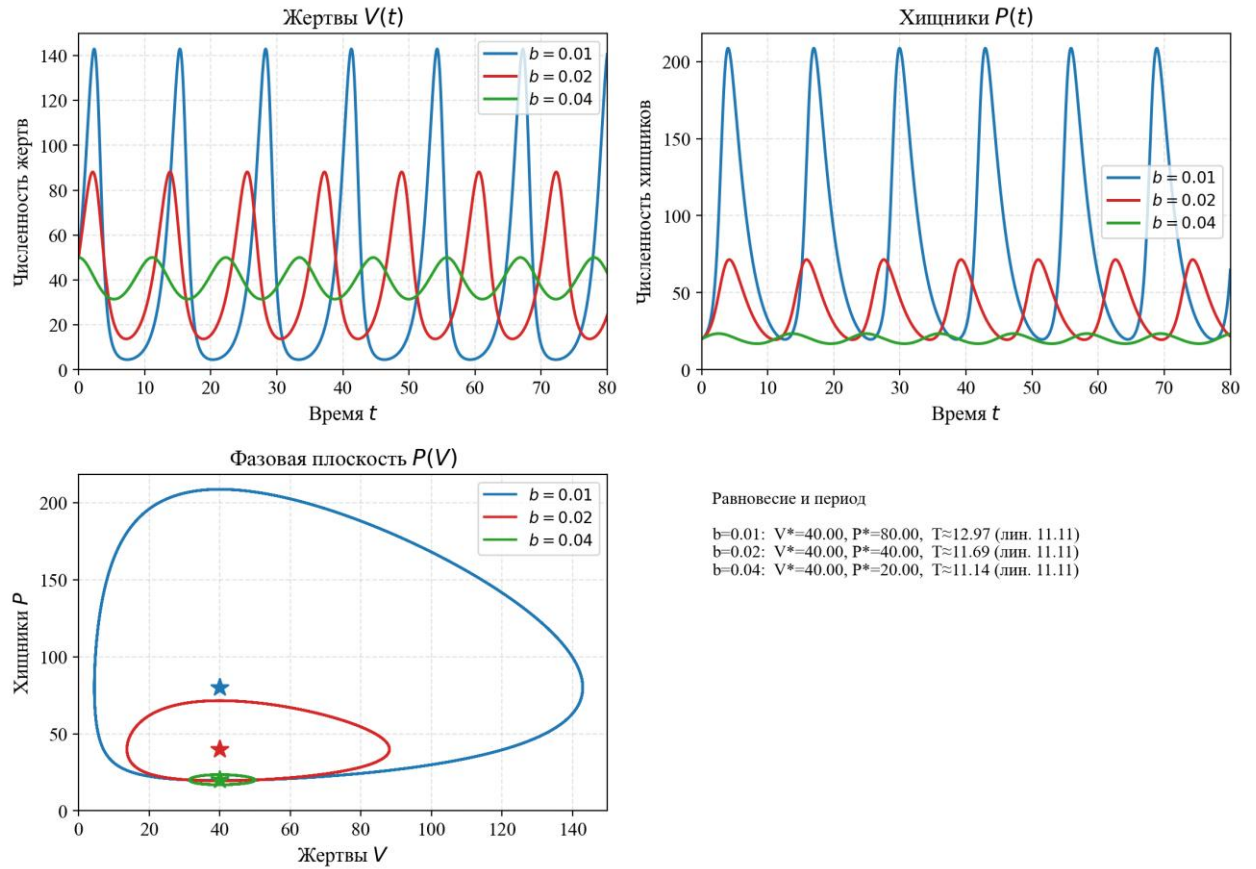
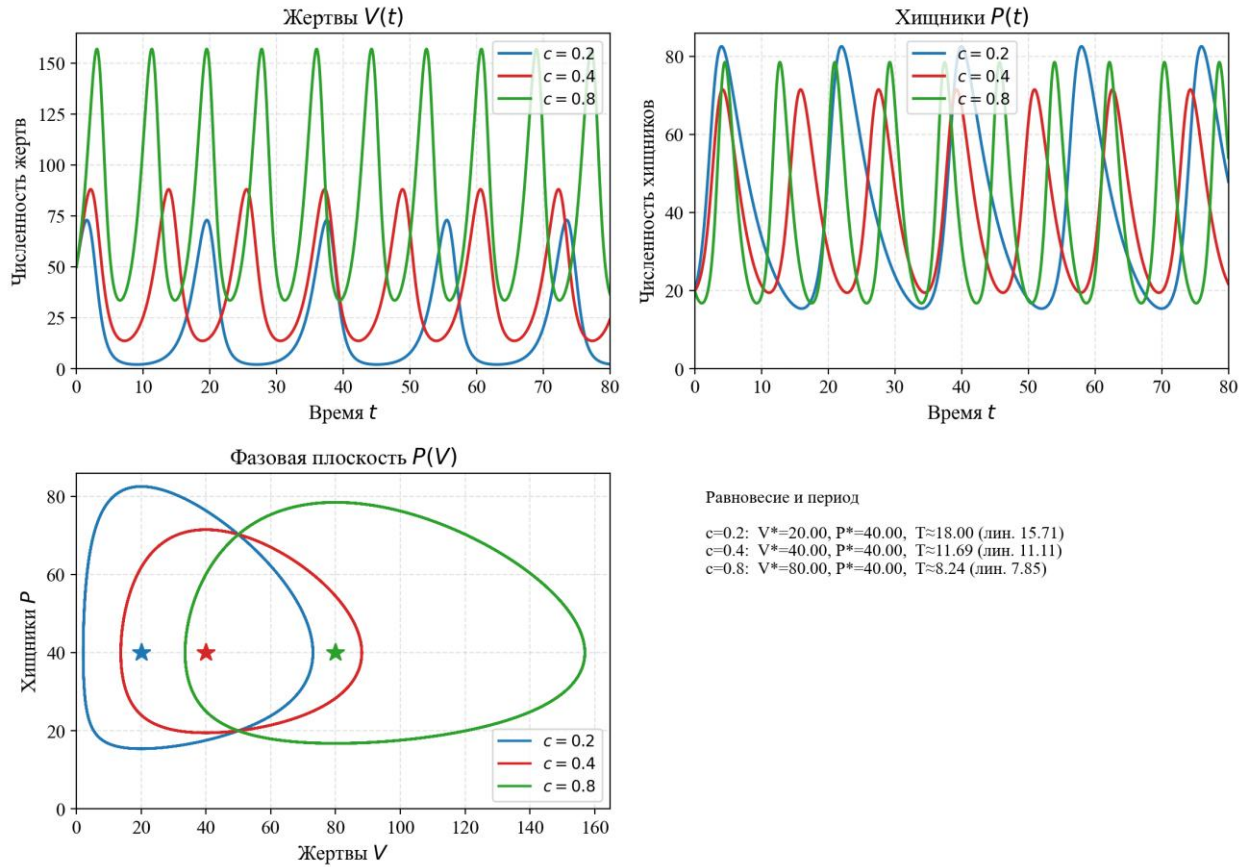


Рис. 6. Влияние коэффициента b (смертность жертв при встрече с хищником)

b	V^*	P^*	T	$V_{\max}$	$T_{\max}$
0,01	40	80	12,98	142,91	208,7
0,02	40	40	11,69	88,12	71,49
0,04	40	20	11,14	50	23,46

Влияние коэффициента c (убыль хищников при нехватке еды)Рис. 7. Влияние коэффициента c (убыль хищников при нехватке еды)

c	V^*	P^*	T	$V_{\max}$	$T_{\max}$
0,2	20	40	18	73,03	82,54
0,4	40	40	11,69	88,12	71,49
0,8	80	40	8,24	157	78,49

Влияние коэффициента d (достаточность еды для размножения хищников)

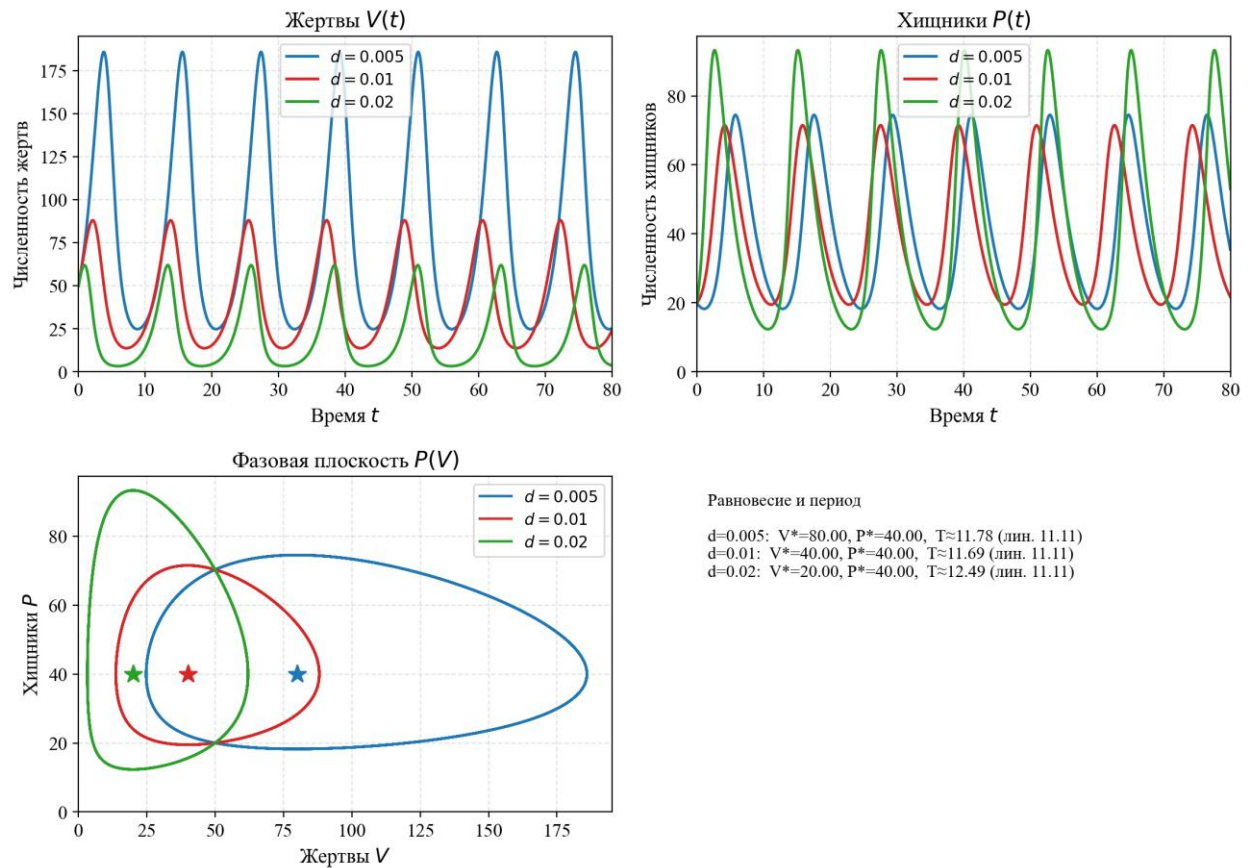


Рис. 8. Влияние коэффициента d (достаточность еды для размножения хищников)

d	V^*	P^*	T	$V_{\max}$	$T_{\max}$
0,005	80	40	11,78	186	74,51
0,01	40	40	11,69	88,12	71,49
0,02	20	40	12,49	62,04	93,26

АНАЛИЗ РЕЗУЛЬТАТОВ И ВЫВОДЫ

Метод Рунге–Кутты 4 порядка с шагом $h = 0,05$ даёт устойчивое решение: погрешность относительно шага $h/2$ порядка $10^{(-6)}$. Этого достаточно для графиков и качественных выводов.

Колебания периодические. Жертвы впереди по фазе: сначала растёт кормовая база V , затем хищники P , они съедают жертв, V падает, затем падает P , цикл повторяется. Пунктир на рис. 2 — уровни равновесия.

Фазовый портрет — замкнутые орбиты вокруг точки $(40; 40)$. Это центр, а не устойчивый фокус: численности не стремятся к точке, а ходят вокруг неё. Чем дальше старт от равновесия, тем больше цикл (рис. 4), центр один и тот же.

Ненулевое равновесие: $(V^*, P^*) = (c/d, a/b)$. Точка $(0, 0)$ неустойчива для жертв: без хищников они растут как $\exp(a \cdot t)$. Сосуществование возможно только во второй точке.

ВЛИЯНИЕ КАЖДОГО КОЭФФИЦИЕНТА

a увеличивается:

жертвы быстрее восстанавливаются, равновесие хищников $P^* = a/b$ растёт, колебания чаще ($T \sim 1/\sqrt{a}$). На фазовой плоскости центр сдвигается вверх (рис. 5).

b увеличивается:

охота успешнее, $P^* = a/b$ падает: тех же жертв «хватает» на меньшее число хищников. Период почти не меняется. Центр сдвигается вниз (рис. 6).

c увеличивается:

хищники быстрее вымирают без еды, жертвам легче, $V^* = c/d$ растёт, колебания чаще. Центр сдвигается вправо (рис. 7).

d увеличивается:

еда лучше идёт в размножение хищников, для их поддержания нужно меньше жертв, $V^* = c/d$ падает. Период почти не меняется. Центр сдвигается влево (рис. 8).

ОБЩИЙ ВЫВОД

Классическая модель Лотки–Вольтерры описывает изолированную систему с периодическими вспышками численности. Компьютерная модель на RK4 воспроизводит замкнутые орбиты, равновесие (c/d , a/b) и влияние всех четырёх коэффициентов. Самоограничения жертв (логистического члена) нет, поэтому амплитуда задаётся начальными условиями и со временем не затухает.

ЛИСТИНГ ПРОГРАММЫ

```

from __future__ import annotations

from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
from matplotlib.patches import FancyArrowPatch, FancyBboxPatch

BASE_DIR = Path(__file__).resolve().parent
FIG_DIR = BASE_DIR / "figures"
RES_DIR = BASE_DIR / "results"

# Базовый набор параметров
A0, B0, C0, D0 = 0.8, 0.02, 0.4, 0.01
V0, P0 = 50.0, 20.0
T0, T_END, H = 0.0, 80.0, 0.05

def lotka_volterra(state: np.ndarray, a: float, b: float, c:
float, d: float) -> np.ndarray:
    """Правая часть системы:  $f(V, P) = [(a - bP)V, (-c + dV)P]$ ."""
    v, p = state
    return np.array([(a - b * p) * v, (-c + d * v) * p],
dtype=float)

def rk4_step(state: np.ndarray, h: float, a: float, b: float, c:
float, d: float) -> np.ndarray:
    """Один шаг классического метода Рунге-Кутты 4-го порядка."""
    k1 = lotka_volterra(state, a, b, c, d)
    k2 = lotka_volterra(state + 0.5 * h * k1, a, b, c, d)
    k3 = lotka_volterra(state + 0.5 * h * k2, a, b, c, d)
    k4 = lotka_volterra(state + h * k3, a, b, c, d)
    return state + (h / 6.0) * (k1 + 2.0 * k2 + 2.0 * k3 + k4)

def rk4_solve(
    v_init: float,
    p_init: float,
    a: float,
    b: float,
    c: float,
    d: float,
    t0: float = T0,
    t_end: float = T_END,
    h: float = H,
) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    """Интегрирование системы на  $[t0, t\_end]$  с шагом  $h$ ."""
    n = int(round((t_end - t0) / h))

```

```

t = t0 + h * np.arange(n + 1, dtype=float)
v = np.empty(n + 1, dtype=float)
p = np.empty(n + 1, dtype=float)
state = np.array([v_init, p_init], dtype=float)
v[0], p[0] = state
for i in range(n):
    state = rk4_step(state, h, a, b, c, d)
    v[i + 1], p[i + 1] = state
return t, v, p

def equilibrium(a: float, b: float, c: float, d: float) ->
tuple[tuple[float, float], tuple[float, float]]:
    """Стационарные точки: (0, 0) и (c/d, a/b)."""
    return (0.0, 0.0), (c / d, a / b)

def conserved_h(v: np.ndarray, p: np.ndarray, a: float, b: float,
c: float, d: float) -> np.ndarray:
    """Первый интеграл классической модели Лотки-Вольтерры."""
    return d * v - c * np.log(v) + b * p - a * np.log(p)

def local_maxima(y: np.ndarray) -> np.ndarray:
    """Индексы строгих локальных максимумов."""
    return np.where((y[1:-1] > y[:-2]) & (y[1:-1] >= y[2:]))[0] +
1

def estimate_period(t: np.ndarray, y: np.ndarray) -> float:
    """Оценка периода по среднему расстоянию между локальными
максимумами."""
    peaks = local_maxima(y)
    if peaks.size < 3:
        return float("nan")
    # первый максимум может быть «обрезан» началом интервала
    return float(np.mean(np.diff(t[peaks[1:]])))

def extrema(y: np.ndarray) -> tuple[float, float]:
    return float(np.max(y)), float(np.min(y))

def step_halving_error(
    v_init: float,
    p_init: float,
    a: float,
    b: float,
    c: float,
    d: float,
    h: float = H,
) -> tuple[float, float]:

```

```

    """Максимальное расхождение решения с шагом h и h/2 (в
    совпадающие моменты)."""
    t_h, v_h, p_h = rk4_solve(v_init, p_init, a, b, c, d, h=h)
    _, v_h2, p_h2 = rk4_solve(v_init, p_init, a, b, c, d, h=h /
2.0)
    v_cmp = v_h2[:, :2]
    p_cmp = p_h2[:, :2]
    n = min(len(t_h), len(v_cmp))
    err_v = float(np.max(np.abs(v_h[:n] - v_cmp[:n])))
    err_p = float(np.max(np.abs(p_h[:n] - p_cmp[:n])))
    return err_v, err_p

```

```

def setup_matplotlib() -> None:
    plt.rcParams.update(
        {
            "font.family": "Times New Roman",
            "font.size": 12,
            "axes.labelsize": 13,
            "axes.titlesize": 14,
            "legend.fontsize": 10,
            "axes.unicode_minus": False,
            "figure.dpi": 120,
            "savefig.dpi": 200,
            "savefig.bbox": "tight",
            "axes.grid": True,
            "grid.alpha": 0.35,
            "grid.linestyle": "--",
        }
    )

```

```

def savefig(fig: plt.Figure, name: str) -> Path:
    path = FIG_DIR / name
    fig.savefig(path, facecolor="white")
    plt.close(fig)
    return path

```

```

def plot_populations_baseline(t, v, p, v_star, p_star) -> None:
    fig, ax = plt.subplots(figsize=(9.5, 5.2))
    ax.plot(t, v, color="#1f77b4", lw=2.0, label="Жертвы $V(t)$")
    ax.plot(t, p, color="#d62728", lw=2.0, label="Хищники $P(t)$")
    ax.axhline(v_star, color="#1f77b4", ls=":", lw=1.3,
label=f"Равновесие жертв $V^*={v_star:.1f}$")
    ax.axhline(p_star, color="#d62728", ls=":", lw=1.3,
label=f"Равновесие хищников $P^*={p_star:.1f}$")
    ax.set_xlabel("Время $t$")
    ax.set_ylabel("Численность")
    ax.set_title("Динамика популяций во времени (базовый набор
параметров)")
    ax.legend(loc="upper right", ncol=2, framealpha=0.95)
    ax.set_xlim(t[0], t[-1])

```

```

ax.set_ylim(bottom=0)
savefig(fig, "01_populations_baseline.png")

def plot_phase_baseline(v, p, v_star, p_star) -> None:
    fig, ax = plt.subplots(figsize=(6.6, 6.4))
    ax.plot(v, p, color="#2ca02c", lw=1.8, label="Траектория  
$(V(t), P(t))$")
    ax.plot(v[0], p[0], "o", color="#1f77b4", ms=8, zorder=5,
label=f"Старт $({v[0]:.0f}, \{p[0]:.0f})$")
    ax.plot(v_star, p_star, "*", color="#d62728", ms=16, zorder=6,
label=f"Равновесие $({v_star:.0f}, \{p_star:.0f})$")
    ax.set_xlabel("Жертвы $V$")
    ax.set_ylabel("Хищники $P$")
    ax.set_title("Фазовая диаграмма (ось $X$ – жертвы, ось $Y$ –  
хищники)")
    ax.legend(loc="best", framealpha=0.95)
    ax.set_xlim(left=0)
    ax.set_ylim(bottom=0)
    savefig(fig, "02_phase_baseline.png")

def plot_phase_family(a, b, c, d, v_star, p_star) -> None:
    initials = [(40.0, 32.0), (50.0, 20.0), (80.0, 20.0)]
    colors = ["#1f77b4", "#2ca02c", "#ff7f0e"]
    fig, ax = plt.subplots(figsize=(6.8, 6.5))
    for (v_init, p_init), color in zip(initials, colors):
        _, v, p = rk4_solve(v_init, p_init, a, b, c, d)
        ax.plot(v, p, color=color, lw=1.7,
label=f"$V_0=\{v_init:.0f\}$, $P_0=\{p_init:.0f\}$")
        ax.plot(v[0], p[0], "o", color=color, ms=6, zorder=5)
        ax.plot(v_star, p_star, "*", color="#d62728", ms=16, zorder=6,
label=f"Равновесие $({v_star:.0f}, \{p_star:.0f})$")
        ax.set_xlabel("Жертвы $V$")
        ax.set_ylabel("Хищники $P$")
        ax.set_title("Семейство фазовых орбит при разных начальных  
условиях")
        ax.legend(loc="upper right", framealpha=0.95)
        ax.set_xlim(left=0)
        ax.set_ylim(bottom=0)
        savefig(fig, "03_phase_family.png")

def plot_coefficient_variation(
    coeff_name: str,
    values: list[float],
    filename: str,
    title: str,
    base: dict[str, float],
) -> list[dict]:
    """Сравнение трёх значений одного коэффициента: время и  
фаза."""
    colors = ["#1f77b4", "#d62728", "#2ca02c"]

```

```

fig, axes = plt.subplots(2, 2, figsize=(11.2, 8.4))
ax_v, ax_p, ax_ph, ax_eq = axes.ravel()
rows: list[dict] = []

for value, color in zip(values, colors):
    params = dict(base)
    params[coeff_name] = value
    a, b, c, d = params["a"], params["b"], params["c"],
params["d"]
    t, v, p = rk4_solve(params["V0"], params["P0"], a, b, c,
d)
    _, (v_star, p_star) = equilibrium(a, b, c, d)
    period = estimate_period(t, v)
    vmax, vmin = extrema(v)
    pmax, pmin = extrema(p)
    rows.append(
        {
            "coeff": coeff_name,
            "value": value,
            "V_star": v_star,
            "P_star": p_star,
            "T_num": period,
            "T_lin": 2.0 * np.pi / np.sqrt(a * c),
            "V_max": vmax,
            "V_min": vmin,
            "P_max": pmax,
            "P_min": pmin,
        }
    )
    label = f"${coeff_name}={value:g}$"
    ax_v.plot(t, v, color=color, lw=1.8, label=label)
    ax_p.plot(t, p, color=color, lw=1.8, label=label)
    ax_ph.plot(v, p, color=color, lw=1.6, label=label)
    ax_ph.plot(v_star, p_star, "*", color=color, ms=12,
zorder=5)

    ax_v.set_title("Жертвы $V(t)$")
    ax_p.set_title("Хищники $P(t)$")
    ax_ph.set_title("Фазовая плоскость $P(V)$")
    for ax, ylab in ((ax_v, "Численность жертв"), (ax_p,
"Численность хищников")):
        ax.set_xlabel("Время $t$")
        ax.set_ylabel(ylab)
        ax.legend(loc="best")
        ax.set_xlim(T0, T_END)
        ax.set_ylim(bottom=0)
    ax_ph.set_xlabel("Жертвы $V$")
    ax_ph.set_ylabel("Хищники $P$")
    ax_ph.legend(loc="best")
    ax_ph.set_xlim(left=0)
    ax_ph.set_ylim(bottom=0)

ax_eq.axis("off")

```

```

lines = ["Равновесие и период\n"]
for row in rows:
    lines.append(
        f"{coeff_name}={row['value']:g}:  "
        f"V*={row['V_star']:.2f}, P*={row['P_star']:.2f},  "
        f"T≈{row['T_num']:.2f} (лин. {row['T_lin']:.2f})"
    )
    ax_eq.text(0.02, 0.95, "\n".join(lines), va="top", ha="left",
family="Times New Roman", fontsize=11)

fig.suptitle(title, fontsize=15)
fig.tight_layout()
savefig(fig, filename)
return rows

def _box(ax, xy, w, h, text, fc="#e8f1fb") -> None:
    x, y = xy
    patch = FancyBboxPatch(
        (x - w / 2, y - h / 2),
        w,
        h,
        boxstyle="round,pad=0.015,rounding_size=0.08",
        linewidth=1.4,
        edgecolor="#1f3b5c",
        facecolor=fc,
    )
    ax.add_patch(patch)
    ax.text(x, y, text, ha="center", va="center", fontsize=10,
wrap=True)

def _diamond(ax, xy, w, h, text) -> None:
    x, y = xy
    xs = [x, x + w / 2, x, x - w / 2]
    ys = [y + h / 2, y, y - h / 2, y]
    ax.fill(xs, ys, facecolor="#fff4d6", edgecolor="#1f3b5c",
linewidth=1.4)
    ax.text(x, y, text, ha="center", va="center", fontsize=10)

def _arrow(ax, start, end) -> None:
    ax.add_patch(
        FancyArrowPatch(
            start,
            end,
            arrowstyle="-|>",
            mutation_scale=12,
            linewidth=1.2,
            color="#1f3b5c",
            shrinkA=0,
            shrinkB=0,
        )
    )

```

```

)

def plot_rk4_flowchart() -> None:
    fig, ax = plt.subplots(figsize=(8.4, 11.4))
    ax.set_xlim(0, 10)
    ax.set_ylim(0, 14.6)
    ax.axis("off")
    ax.set_title("Блок-схема: решение системы методом Рунге-Кутты
4 порядка", pad=12)

    _box(ax, (5, 13.55), 5.2, 0.8, "Начало", fc="#d9f2d9")
    _box(ax, (5, 12.35), 7.2, 0.95, "Ввод: a, b, c, d, V0, P0, t0,
T, h")
    _box(ax, (5, 11.05), 7.2, 0.95, "t ← t0, V ← V0, P ←
P0\nсохранить (t, V, P)")
    _diamond(ax, (5, 9.45), 4.6, 1.25, "t ≥ T ?")
    _box(
        ax,
        (5, 7.25),
        7.6,
        2.05,
        "k1 = f(V, P)\n"
        "k2 = f(V + h k1/2, P + h k1/2)\n"
        "k3 = f(V + h k2/2, P + h k2/2)\n"
        "k4 = f(V + h k3, P + h k3)",
        fc="#f7e6ff",
    )
    _box(
        ax,
        (5, 5.15),
        7.6,
        1.35,
        "V ← V + h(k1 + 2k2 + 2k3 + k4)/6\n"
        "P ← P + h(k1 + 2k2 + 2k3 + k4)/6\n"
        "t ← t + h; сохранить (t, V, P)",
    )
    _box(ax, (5, 3.35), 7.2, 1.05, "Графики V(t), P(t) и фазовая
диаграмма P(V)")
    _box(ax, (5, 2.05), 7.2, 1.0, "Стационарная точка (c/d,
a/b)\nни анализ коэффициентов")
    _box(ax, (5, 0.8), 5.2, 0.8, "Конец", fc="#d9f2d9")

    _arrow(ax, (5, 13.15), (5, 12.85))
    _arrow(ax, (5, 11.87), (5, 11.55))
    _arrow(ax, (5, 10.55), (5, 10.1))
    _arrow(ax, (5, 8.82), (5, 8.3))
    ax.text(5.25, 8.45, "нет", fontsize=10, color="#1f3b5c")
    _arrow(ax, (5, 6.2), (5, 5.85))

    # возврат к проверке t ≥ T (слева)

```

```

    ax.plot([5 - 3.8, 1.15], [5.15, 5.15], color="#1f3b5c",
lw=1.2)
    ax.annotate(
        "",
        xy=(1.15, 9.45),
        xytext=(1.15, 5.15),
        arrowprops=dict(arrowstyle="->", color="#1f3b5c",
lw=1.2),
    )
    ax.plot([1.15, 5 - 2.3], [9.45, 9.45], color="#1f3b5c",
lw=1.2)

    # ветка «да» (справа)
    ax.text(7.45, 9.65, "да", fontsize=10, color="#1f3b5c")
    ax.plot([5 + 2.3, 8.85], [9.45, 9.45], color="#1f3b5c",
lw=1.2)
    ax.annotate(
        "",
        xy=(8.85, 3.35),
        xytext=(8.85, 9.45),
        arrowprops=dict(arrowstyle="->", color="#1f3b5c",
lw=1.2),
    )
    ax.plot([8.85, 5 + 3.6], [3.35, 3.35], color="#1f3b5c",
lw=1.2)

    _arrow(ax, (5, 2.8), (5, 2.58))
    _arrow(ax, (5, 1.52), (5, 1.22))

    fig.tight_layout()
    savefig(fig, "rk4_flowchart.png")

def write_csv(t, v, p) -> None:
    path = RES_DIR / "baseline.csv"
    with path.open("w", encoding="utf-8", newline="") as f:
        f.write("t;V_pre;P_predator\n")
        for ti, vi, pi in zip(t, v, p):
            f.write(f"{ti:.6f};{vi:.8f};{pi:.8f}\n")

def write_summary(stats: dict, vary_rows: list[dict]) -> None:
    s = stats
    text = f""""Модель Лотки-Вольтерры, метод Рунге-Кутты 4 порядка
=====

Система уравнений

$$\begin{aligned} dV/dt &= (a - b P) V && (V - \text{жертвы}) \\ dP/dt &= (-c + d V) P && (P - \text{хищники}) \end{aligned}$$


Базовые параметры
a = {s['a']:g}
b = {s['b']:g}

```

```

c = {s['c']:g}
d = {s['d']:g}
V(0) = {s['V0']:g}
P(0) = {s['P0']:g}
t in [{s['t0']:g}, {s['t_end']:g}], h = {s['h']:g}, шагов =
{s['n_steps']}

```

Стационарные точки (общий вид)

```

(V, P) = (0, 0)
(V*, P*) = (c/d, a/b)

```

Стационарные точки (базовый набор)

```

(0, 0)
(V*, P*) = ({s['V_star']:.8f}, {s['P_star']:.8f})

```

Экстремумы на $[0, \{s['t_end']:g\}]$

```

жертвы: V_max = {s['V_max']:.6f}, V_min = {s['V_min']:.6f}
хищники: P_max = {s['P_max']:.6f}, P_min = {s['P_min']:.6f}

```

Период колебаний

```

численно (по максимумам V): T ≈ {s['T_num']:.6f}
линейная оценка  $2\pi/\sqrt{a \ c}$ : T ≈ {s['T_lin']:.6f}

```

Контроль точности RK4 (сравнение шагов h и h/2)

```

max |V_h - V_{h/2}| = {s['err_v']:.3e}
max |P_h - P_{h/2}| = {s['err_p']:.3e}

```

Первый интеграл $H = d \ V - c \ \ln V + b \ P - a \ \ln P$

```

H(0) = {s['H0']:.8f}
max|H-H0| = {s['H_drift']:.3e}

```

(малый дрейф H подтверждает, что RK4 хорошо держит замкнутые орбиты)

Вариации коэффициентов

```

"""
    for row in vary_rows:
        text += (
            f" {row['coeff']}={row['value']:g}: "
            f"V*={row['V_star']:.4f}, P*={row['P_star']:.4f}, "
            f"T≈{row['T_num']:.4f} (лин. {row['T_lin']:.4f}), "
            f"V∈[{row['V_min']:.2f}; {row['V_max']:.2f}], "
            f"P∈[{row['P_min']:.2f}; {row['P_max']:.2f}]\n"
        )
    (RES_DIR / "summary.txt").write_text(text, encoding="utf-8")

```

def main() -> None:

```

FIG_DIR.mkdir(exist_ok=True)
RES_DIR.mkdir(exist_ok=True)
setup_matplotlib()

```

```

a, b, c, d = A0, B0, C0, D0
t, v, p = rk4_solve(V0, P0, a, b, c, d)
_, (v_star, p_star) = equilibrium(a, b, c, d)

```

```

t_lin = 2.0 * np.pi / np.sqrt(a * c)
t_num = estimate_period(t, v)
vmax, vmin = extrema(v)
pmax, pmin = extrema(p)
err_v, err_p = step_halving_error(V0, P0, a, b, c, d)
h_hist = conserved_h(v, p, a, b, c, d)
stats = {
    "a": a,
    "b": b,
    "c": c,
    "d": d,
    "V0": V0,
    "P0": P0,
    "t0": T0,
    "t_end": T_END,
    "h": H,
    "n_steps": len(t) - 1,
    "V_star": v_star,
    "P_star": p_star,
    "V_max": vmax,
    "V_min": vmin,
    "P_max": pmax,
    "P_min": pmin,
    "T_num": t_num,
    "T_lin": t_lin,
    "err_v": err_v,
    "err_p": err_p,
    "H0": float(h_hist[0]),
    "H_drift": float(np.max(np.abs(h_hist - h_hist[0]))),
}

plot_populations_baseline(t, v, p, v_star, p_star)
plot_phase_baseline(v, p, v_star, p_star)
plot_phase_family(a, b, c, d, v_star, p_star)
plot_rk4_flowchart()

base = {"a": a, "b": b, "c": c, "d": d, "V0": V0, "P0": P0}
vary_rows: list[dict] = []
vary_rows += plot_coefficient_variation(
    "a",
    [0.4, 0.8, 1.2],
    "04_vary_a.png",
    "Влияние коэффициента $a$ (рождаемость жертв)",
    base,
)
vary_rows += plot_coefficient_variation(
    "b",
    [0.01, 0.02, 0.04],
    "05_vary_b.png",
    "Влияние коэффициента $b$ (смертность жертв при встрече с хищником)",
    base,
)

```

```

vary_rows += plot_coefficient_variation(
    "c",
    [0.2, 0.4, 0.8],
    "06_vary_c.png",
    "Влияние коэффициента $c$ (убыль хищников при нехватке
еды)",
    base,
)
vary_rows += plot_coefficient_variation(
    "d",
    [0.005, 0.01, 0.02],
    "07_vary_d.png",
    "Влияние коэффициента $d$ (достаточность еды для
размножения хищников)",
    base,
)

write_csv(t, v, p)
write_summary(stats, vary_rows)

print("Готово.")
print(f"  графики: {FIG_DIR}")
print(f"  результаты: {RES_DIR}")
print(f"  равновесие: V*={v_star:.4f}, P*={p_star:.4f}")
print(f"  период: T≈{t_num:.4f} (лин. {t_lin:.4f})")
print(f"  погрешность h vs h/2: dV={err_v:.3e},
dP={err_p:.3e}")

if __name__ == "__main__":
    main()

```